

人工智能引论

强化学习

北京邮电大学人工智能引论教研团队



北京邮电大学
Beijing University of Posts and Telecommunications



本次课的主要内容



北京邮电大学
Beijing University of Posts and Telecommunications

强化学习基本概念

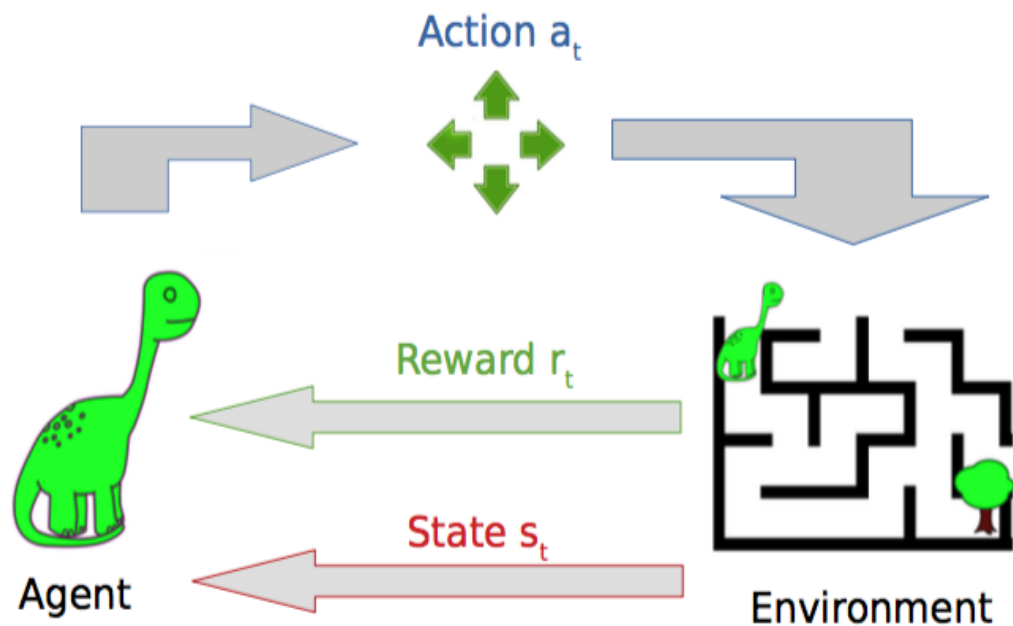
基于值的强化学习

基于策略的强化学习

法

强化学习的定义

- **强化学习**（Reinforcement Learning）是机器学习的一个分支，相较于机器学习经典的有监督学习、无监督学习问题，强化学习最大的特点是在**交互中学习**。
- **智能体**（Agent）通过与环境的交互来学习策略，以最大化累积奖励。

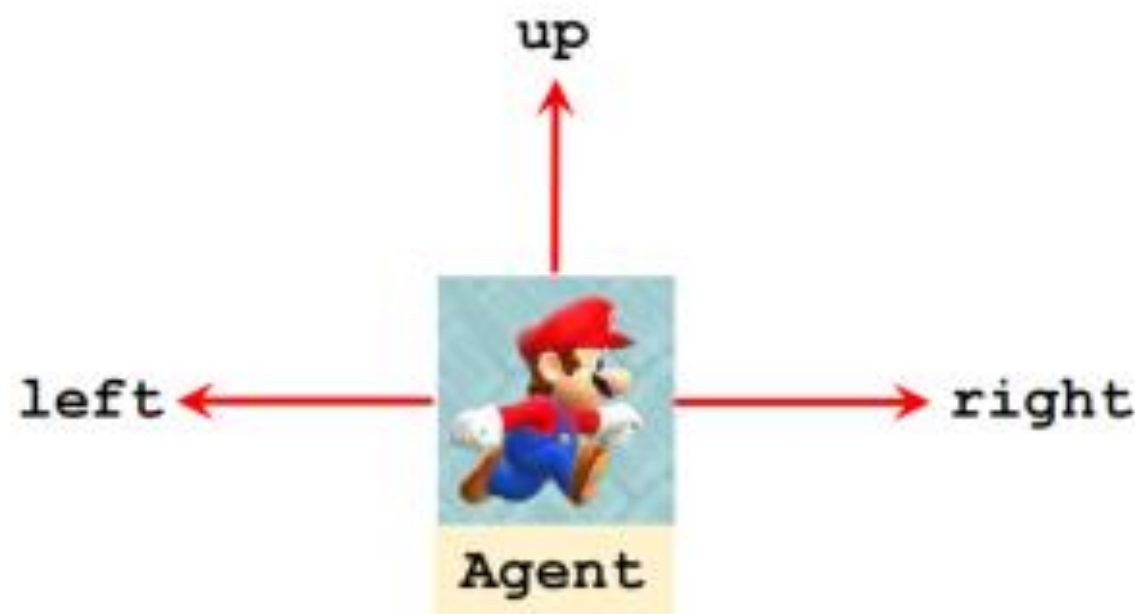


强化学习基本概念：状态与动作

状态(state): s (当前帧)



动作 (Action) : $a \in \{left, right, up\}$



强化学习基本概念：策略

策略 (policy) : π

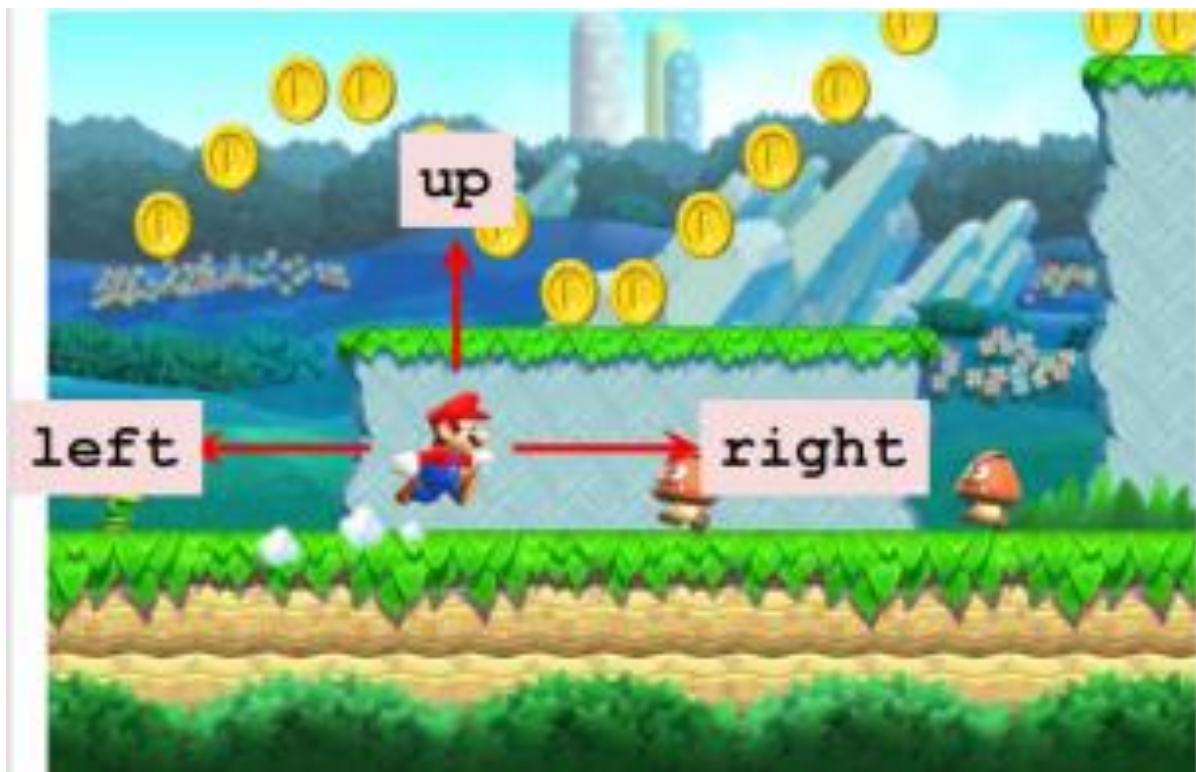
➤ 策略函数 $\pi: (s, a) \rightarrow [0,1]$:

$$\pi(a|s) = P(A = a|S = s)$$

➤ 例子：给出状态 s ，作出动作 a 的概率是：

- $\pi(left | s) = 0.2,$
- $\pi(right | s) = 0.1,$
- $\pi(up | s) = 0.7.$

➤ 一旦观察 state $S = s$ ，agent 的动作 action A 可以是确定的，也可以是随机的

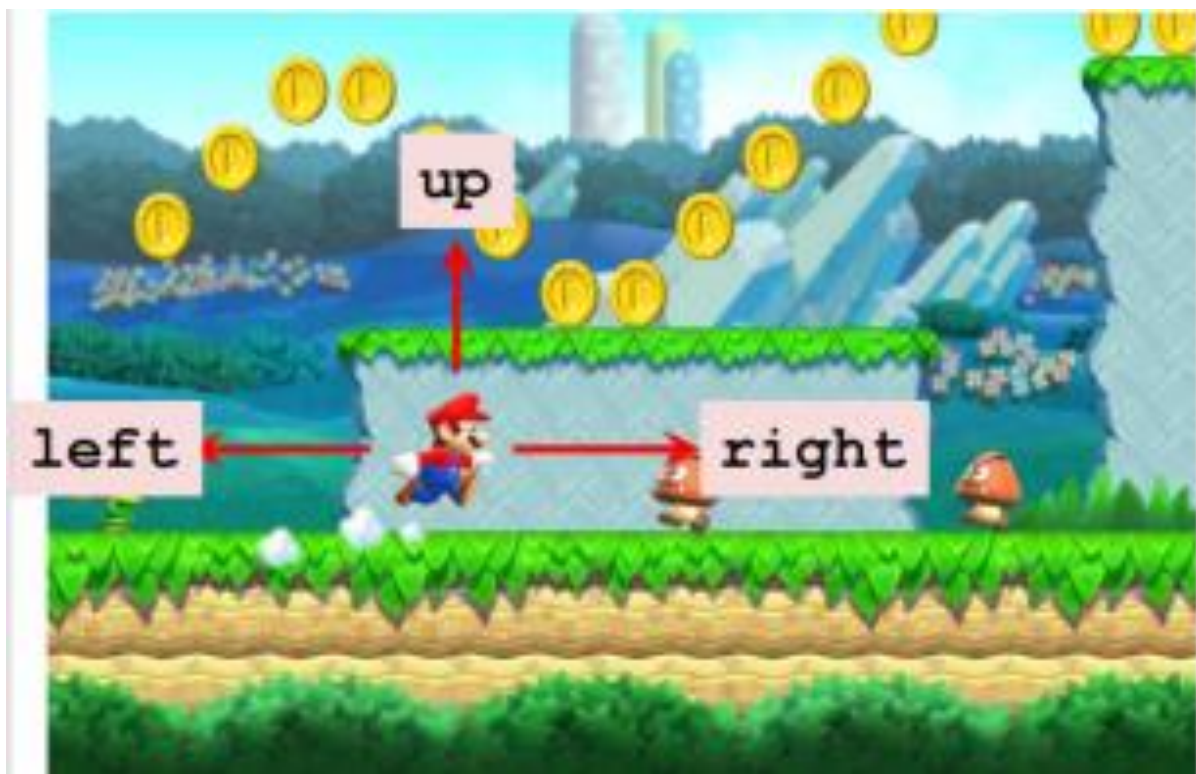


强化学习基本概念：策略



北京邮电大学
Beijing University of Posts and Telecommunications

随机策略 or 确定性策略？



强化学习基本概念：奖励Reward



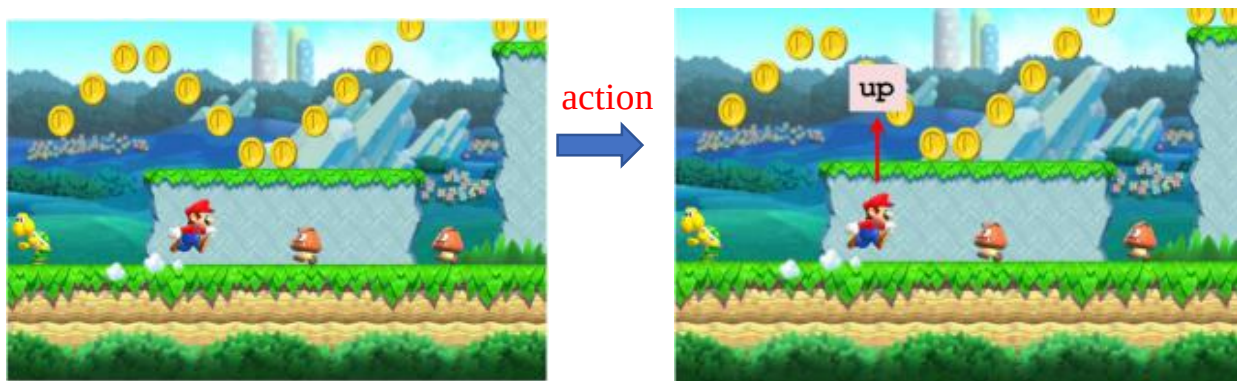
北京邮电大学
Beijing University of Posts and Telecommunications

reward R



- 收集到硬币: $R = +1$
- 赢了游戏: $R = +10000$
- 输了游戏: $R = -10000$
- 没有事情发生: $R = 0$

强化学习基本概念：状态转移



state transition



- 例如，“up”动作导致了Agent 进入新的状态
- 状态转移可以是确定的也可以是随机的；
- 通常我们认为状态转移是随机的，这种随机性是来自于环境

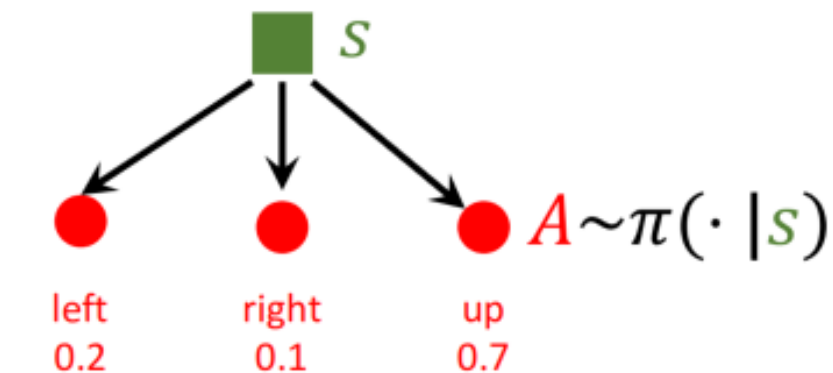


$$(s' | s, a) = P(S' = s' | S = s, A = a)$$

强化学习的随机性

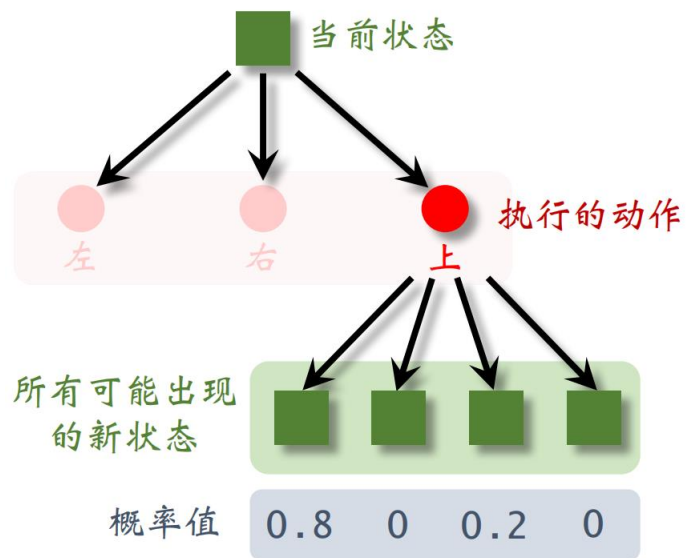
■ 动作的随机性

- 给定当前状态 s , 策略函数 $\pi(a|s)$ 会算出动作空间 A 中每个动作 a 的概率值。
- 动作的随机性来自于**随机决策**: $A \sim \pi(\cdot | s)$.



■ 状态的随机性

- 当状态 s 和动作 a 都被确定下来, 下一个状态仍然有随机性。
- 状态的随机性来自于**状态转移函数**: $s' \sim p(\cdot | s, a)$.

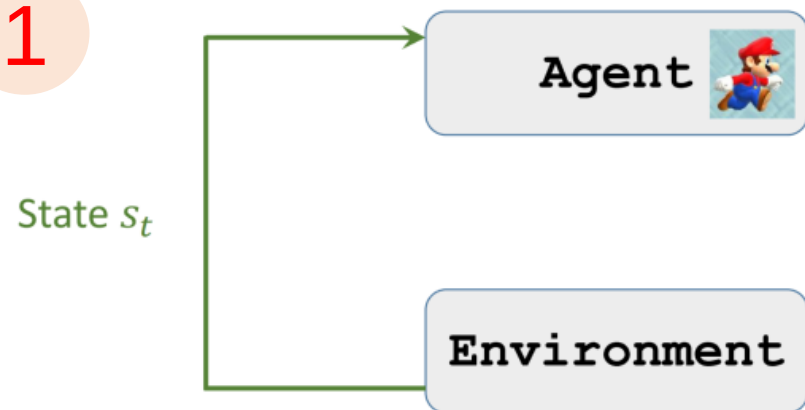


基本概念：agent与环境交互

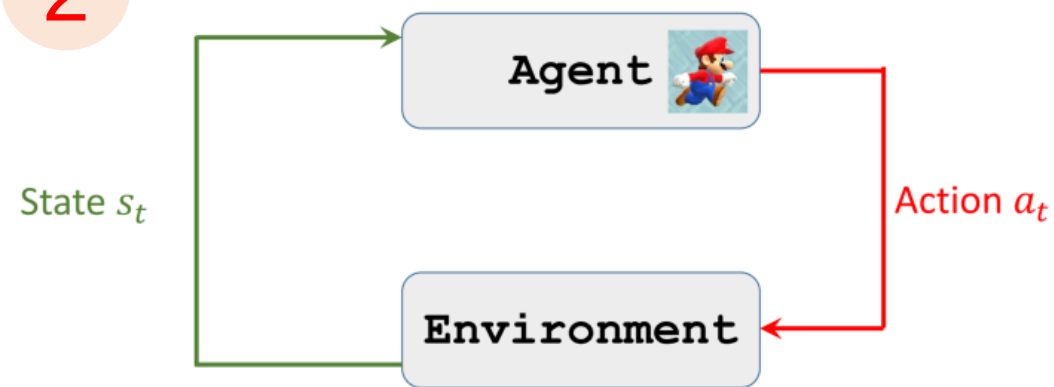


北京邮电大学
Beijing University of Posts and Telecommunications

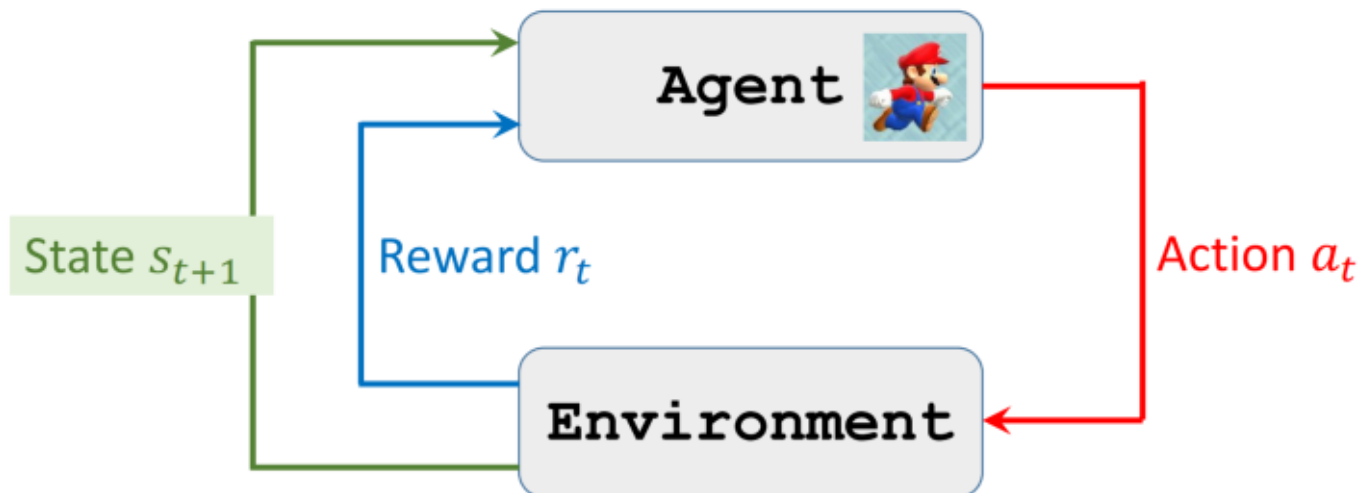
1



2



3

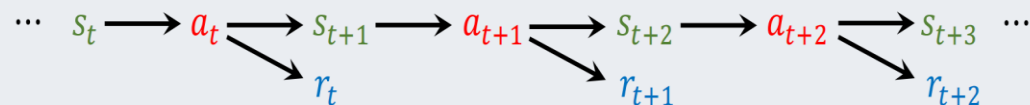


Play the game using AI



- 观察当前帧frame (state s_1)
 - → 执行动作 action a_1 (left, right, or up)
 - → 观察新的帧 (state s_2) 和奖励reward r_1
 - → 执行动作 action a_2
 - →
-
- 轨迹 (trajectory) 是指一回合 (episode) 游戏中, 智能体观测到的所有的状态、动作、奖励 (state, action, reward):

$$s_1, a_1, r_1, s_2, a_2, r_2, s_3, a_3, r_3, \dots s_T, a_T, r_T.$$



智能体的轨迹

- 回合 (episode): 游戏从开始到结束 (Mario 赢 or 死)

Play the game using AI

问题：什么是好的策略？

基本概念：reward and return

回报（Return）：是从当前时刻开始到本回合结束的所有奖励的总和,所以回报也叫做累计奖励（cumulative future reward）

➤ 回报有什么用呢？

- 回报是未来获得的奖励总和，所以智能体的目标就是让回报尽量大，越大越好。
- 强化学习的目标就是寻找一个策略，使得回报的期望最大化。这个策略称为最优策略（optimum policy）。

注：强化学习的目标是最大化回报，而不是最大化当前的奖励。
奖励是策略学习的指导

基本概念：reward and return

回报（Return）：是从当前时刻开始到本回合结束的所有奖励的总和,所以回报也叫做累计奖励（cumulative future reward）

$$\bullet U_t = R_t + R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

问题：在时刻 t , 奖励 R_t 和 R_{t+1} 是一样重要吗？

➤ 假如我给你两个选项，你选哪一个？
第一，现在我立刻给你 100 元钱；
第二，等一年后我给你 100 元钱。

➤ 换一个，你选哪一个？
第一，现在我立刻给你 80 元钱；
第二，等一年后我给你 100 元钱。

- 如果两种选择一样好，未来奖励对当前的重要性要少于当前的奖励
- R_{t+1} 的重要性应该少于 R_t

基本概念：折扣回报 (discounted returns)

回报 (Return)：是从当前时刻开始到本回合结束的所有奖励的总和，所以回报也叫做累计奖励 (cumulative future reward)

$$\bullet U_t = R_t + R_{t+1} + R_{t+2} + R_{t+3} + \dots$$

折扣回报 (Return)：给未来的奖励做折扣

- γ ：折扣率 (discount factor)

$$U_t = R_t + \gamma \cdot R_{t+1} + \gamma^2 \cdot R_{t+2} + \gamma^3 \cdot R_{t+3} + \dots$$

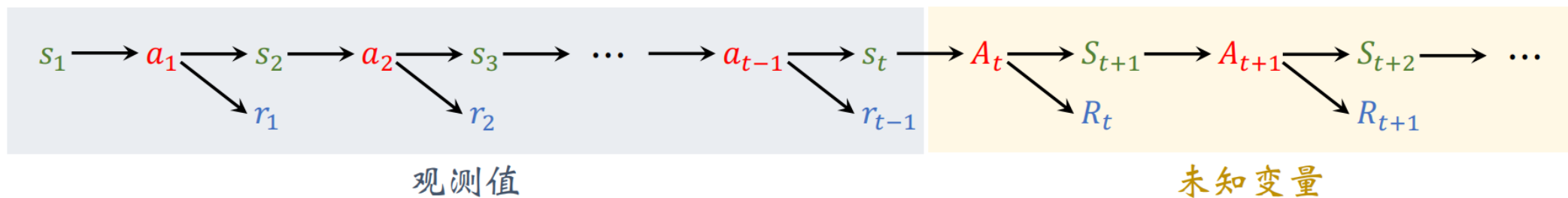
基本概念：折扣回报的随机性

- 假设我们此时在第 t 时刻，只观测到 s_t 及其之前的状态、动作、奖励

$s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_{t-1}, a_{t-1}, r_{t-1}, s_t,$

- 而下面这些都是随机变量（尚未被观测到）：

$A_t, R_t, s_{t+1}, A_{t+1}, R_{t+1}, \dots, s_n, A_n, R_n.$



- 回报 U_t 依赖于奖励 $R_t, R_{t+1}, \dots, R_n$ ，而这些奖励全都是未知的随机变量，所以 U_t 也是未知的随机变量。

基本概念：动作价值函数 $Q(s, a)$

➤ 折扣回报

$$U_t = R_t + \gamma \cdot R_{t+1} + \gamma^2 \cdot R_{t+2} + \gamma^3 \cdot R_{t+3} + \dots$$

➤ 策略 π 的动作价值函数：

$$Q_{\pi}(s_t, a_t) = \mathbb{E}_{S_{t+1}, A_{t+1}, \dots, S_n, A_n} \left[U_t \mid S_t = s_t, A_t = a_t \right].$$

➤ 回报 U_t 依赖于动作 $A_t, A_{t+1}, A_{t+2}, \dots$ 和状态 $S_t, S_{t+1}, S_{t+2}, \dots$

- 动作随机性： $P[A = a | S = s] = \pi(a | s)$

策略函数

- 状态随机性： $P[s' = s' | S = s, A = a] = p(s' | s, a)$

状态转移函数



基本概念：动作价值函数 $Q(s, a)$

➤ 折扣回报

$$U_t = R_t + \gamma \cdot R_{t+1} + \gamma^2 \cdot R_{t+2} + \gamma^3 \cdot R_{t+3} + \dots$$

➤ 策略 π 的动作价值函数：

$$Q_{\pi}(s_t, a_t) = \mathbb{E}_{S_{t+1}, A_{t+1}, \dots, S_n, A_n} \left[U_t \mid S_t = s_t, A_t = a_t \right].$$

➤ 最优动作-价值函数

$$Q_{\star}(s_t, a_t) = \max_{\pi} Q_{\pi}(s_t, a_t),$$



基本概念：动作价值函数 $Q(s, a)$

➤ 折扣回报

$$U_t = R_t + \gamma \cdot R_{t+1} + \gamma^2 \cdot R_{t+2} + \gamma^3 \cdot R_{t+3} + \dots$$

➤ 策略 π 的动作价值函数：

$$Q_{\pi}(s_t, a_t) = \mathbb{E}_{S_{t+1}, A_{t+1}, \dots, S_n, A_n} \left[U_t \mid S_t = s_t, A_t = a_t \right].$$

➤ 状态价值函数：state-value function

- $V_{\pi}(s_t) = \mathbb{E}_A [Q_{\pi}(s_t, A)] = \sum_a \pi(a|s_t) \cdot Q_{\pi}(s_t, a)$. (Actions are discrete.)
- $V_{\pi}(s_t) = \mathbb{E}_A [Q_{\pi}(s_t, A)] = \int \pi(a|s_t) \cdot Q_{\pi}(s_t, a) da$. (Actions are continuous.)



总结：价值函数

- 动作价值函数： $Q_{\pi}(s, a) = E[U_t | S_t = s, A_t = a]$
- 给定策略 π ， $Q_{\pi}(s, a)$ 评估在当前状态 s 下，选择动作 a 有多好。
- 状态价值函数： $V_{\pi}(s) = E_A[Q_{\pi}(s, A)]$
- 给定策略 π ， $V_{\pi}(s)$ 评估当前所处的状态有多好。

本次课的主要内容



北京邮电大学
Beijing University of Posts and Telecommunications

强化学习基本概念

基于值的强化学习

基于策略的强化学习

法

动作价值函数的近似

■ 目标：赢得游戏 (最大化总的奖励)

■ 问题：如果我们知道了 $Q^*(s, a)$, 最佳的动作是什么？

➤ 显然，最佳的动作是： $a^* = \underset{a}{\operatorname{argmax}} Q^*(s, a)$

Q^* 是一个指示器表征在状态 s 选择动作 a 的好坏。

动作价值函数的近似

■ 目标：赢得游戏 (最大化总的奖励)

■ 问题：如果我们知道了 $Q^*(s, a)$, 最佳的动作是什么？

➤ 显然，最佳的动作是： $a^* = \underset{a}{\operatorname{argmax}} Q^*(s, a)$

■ 挑战：我们不知道 $Q^*(s, a)$ ？

➤ 方法1：Q-learning

➤ 方法2：Deep Q Network (DQN)，采用神经网络 $Q(s, a; w)$ 对 $Q^*(s, a)$ 进行近似。

Q-learning

- Q-learning：一种经典的强化学习算法，基于值迭代的思想，通过估计每个状态动作对的价值函数 Q 来选择每个状态的最佳的动作。

- **用表格表示 Q_*** ：假设状态空间 S 和动作空间 A 都是有限集，即集合中元素数量有限。
- 比如， S 中一共有 3 种状态， A 中一共有 4 种动作。那么最优动作价值函数 $Q_*(s, a)$ 可以表示为一个 3×4 的表格，比如右边的表格。基于当前状态 s_t ，做决策时使用的公式

$$a_t = \operatorname{argmax}_{a \in A} Q_*(s_t, a)$$

	第 1 种 动作	第 2 种 动作	第 3 种 动作	第 4 种 动作
第 1 种 状态	380	-95	20	173
第 2 种 状态	-7	64	-195	210
第 3 种 状态	152	72	413	-80

图 4.5: 最优动作价值函数 Q_* 表示成表格形式。

Q-learning

■ 问题：如何估计价值函数 Q ？

➤ 贝尔曼方程

$$Q^*(s, a) = \mathbb{E}[R_t + \gamma \max_{a'} Q^*(S_{t+1}, a') | S_t = s, A_t = a]$$

其中 s 是当前状态， a 是采取的动作， R 是动作 a 的奖励； s' 是下一个状态， a' 是状态 s' 下的动作。公式中的 $\max Q(s', a')$ 表示：下一状态 s' 的所有动作 a' 中值函数 $Q(s', a')$ 最大的。

Richard S. Sutton and Andrew G. Barto, Reinforcement Learning: An Introduction. 2012.

Q-learning关键在于如何更新Q值

Q-learning 的学习过程:

1. 初始化Q值表: 通常将所有的Q值初始化为零或小的随机值。
2. 选择动作: 根据当前状态选择一个动作, 通常使用 ϵ -贪婪策略。该策略在探索和利用之间进行权衡, 以确保智能体能够探索新的状态和动作。
3. 执行动作: 在环境中执行所选动作, 观察结果状态和获得的奖励。
4. 更新Q值: 根据获得的奖励和下一个状态的最大Q值更新当前状态的Q值。更新公式为:

$$Q(s, a) = (1 - \alpha) * Q(s, a) + \alpha * (r + \gamma * \max_{a'} Q(s', a'))$$

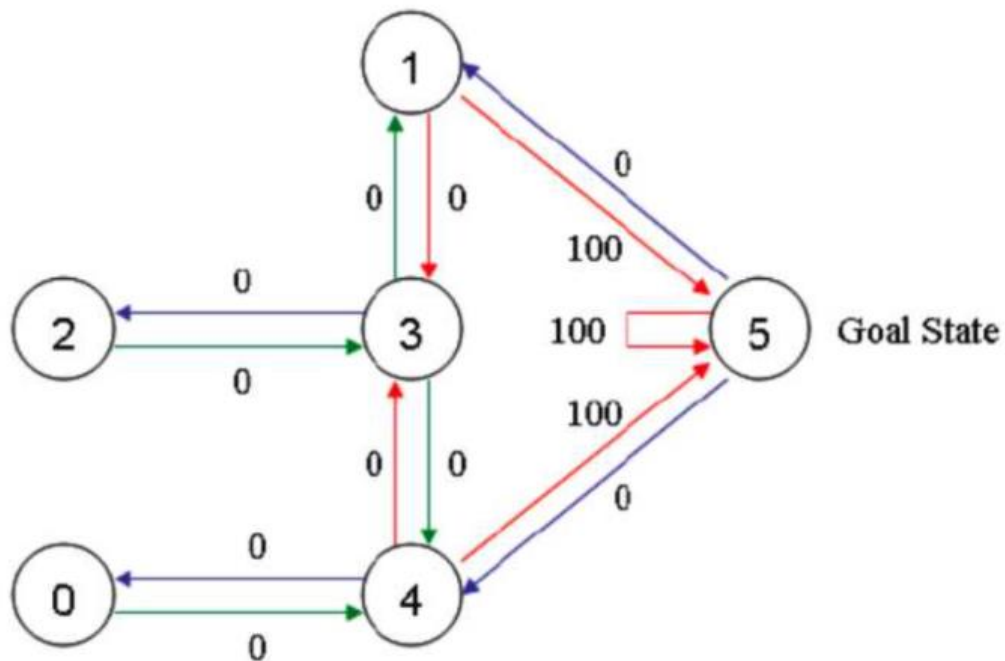
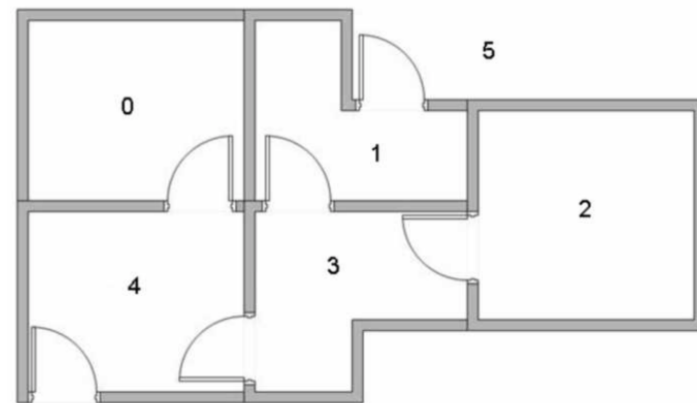
其中 α 是学习率, 控制新信息对旧信息的影响程序。

5. 迭代: 重复上述步骤, 直到Q值收敛或达到预设的迭代次数,

Q-learning例子



假设我们有这样一个房间，我们首先将agent（机器人）处于任何一个位置，让他自己走动，直到走到5房间，表示成功。为了能够走出去，我们将每个节点之间设置一定的权重，能够直接到达5的边设置为100，其他不能的设置0，这样网络的图为：



		Action					
State		0	1	2	3	4	5
0		-1	-1	-1	-1	0	-1
1		-1	-1	-1	0	-1	100
2		-1	-1	-1	0	-1	-1
3		-1	0	0	-1	0	-1
4		0	-1	-1	0	-1	100
5		-1	0	-1	-1	0	100

Q-learning例子

1. 首先创建一个Q表，表示学习到的经验，初始化为0矩阵，表示从一个state到另一个state能获得的总的奖励的折现值。

$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

$$R = \begin{matrix} & \begin{matrix} \text{Action} \\ 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} \text{State} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} -1 & -1 & -1 & -1 & 0 & -1 \\ -1 & -1 & -1 & 0 & -1 & 100 \\ -1 & -1 & -1 & 0 & -1 & -1 \\ -1 & 0 & 0 & -1 & 0 & -1 \\ 0 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & -1 & -1 & 0 & 100 \end{bmatrix} \end{matrix}$$

2. 随机选择一个状态，比如1，查看状态1所对应的R表，也就是1可以到达3或5，随机地，我们选择5，根据转移方程：

$$\begin{aligned} Q(1,5) &= (1 - 0.5) * Q(1,5) + 0.5 * (R(1,5) + 0.8 * \max\{Q(5,1), Q(5,4), Q(5,5)\}) \\ &= 0.5 * 0 + 0.5 * (100 + 0.8 * \max\{0,0,0\}) \\ &= 50 \end{aligned}$$

$$Q = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 50 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Q-learning例子

这样，到达目标，一次尝试结束。接下来再选择一个随机状态，比如3，3对应的下一个状态有（1，2，4都是状态3对应的非负状态），随机地，我们选择1，这样根据算法更新：

$$\begin{aligned} Q(3,1) &= (1 - 0.5) * Q(3,1) + 0.5 * (R(3,1) + 0.8 * \max\{Q(1,3), Q(1,5)\}) \\ &= 0.5 * 0 + 0.5 * (0 + 0.8 * \max\{0, 50\}) \\ &= 40 \end{aligned}$$

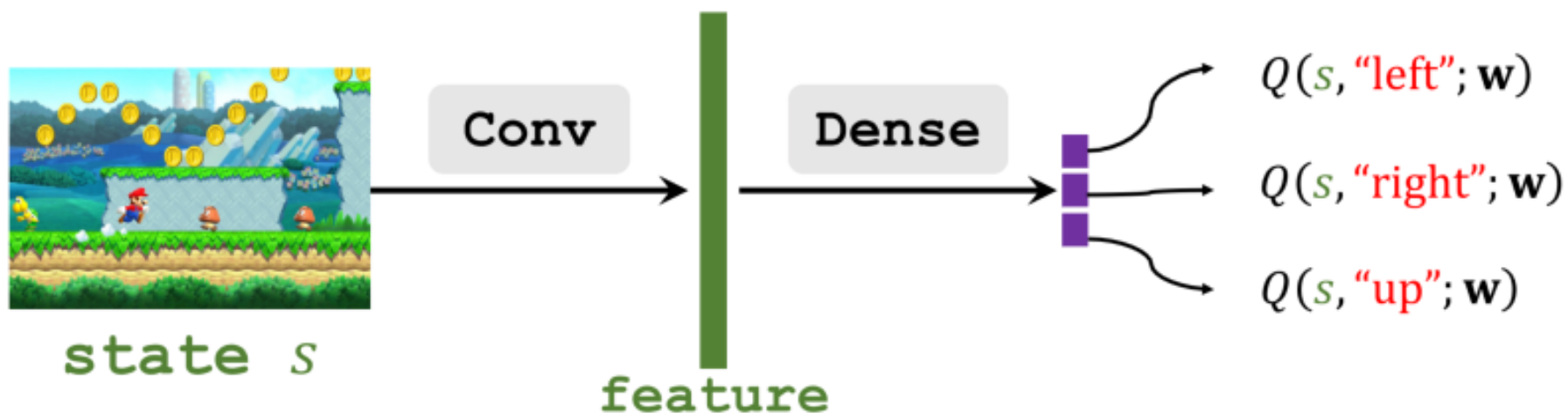
Q表更新为：

$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 50 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 40 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

上述过程不断的迭代，就会学习到一个最终的Q表

Deep Q Network (DQN)

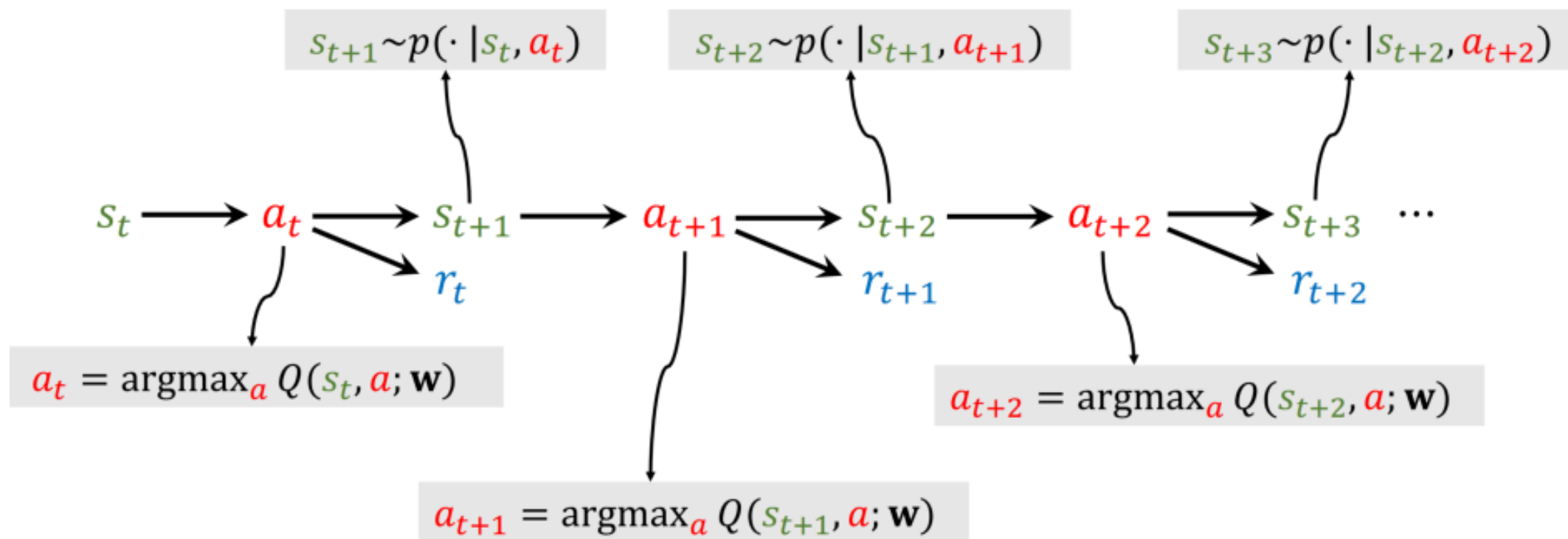
- Deep Q Network (DQN): DQN 结合了深度学习和Q-learning的算法，旨在解决Q-learning在高维状态空间和连续动作空间中的应用限制。
- DQN使用神经网络来近似Q 值函数
 - 输入: size of the screenshot.
 - 输出: 对所有动作a的打分。



学习的目标是：对于所有的 s 和 a ，DQN 的预测 $Q(s, a; w)$ 尽量接近 $Q^*(s, a)$ 。其中 w 表示神经网络中的参数。

DQN如何应用于玩游戏？

- **Question:** Based on the predictions, what should be the **action** ?



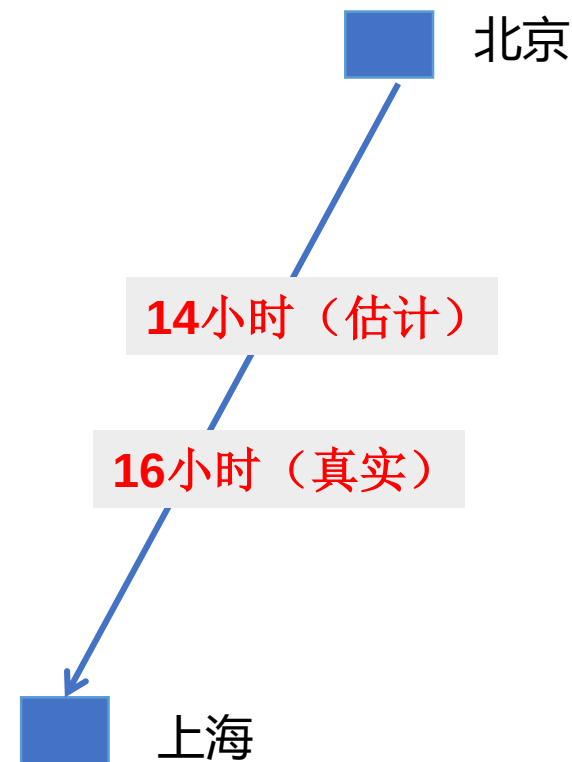
如何训练DQN--TD learning

Temporal difference (TD) learning: 时间差分算法

- 举例：假设我是个用户，我要从北京驾车去上海。
- 模型 $Q(w)$ 估计总车程：例如是 14 小时：

问题：如何更新模型？

- 做预测： $q = Q(w)$, e.g., $q = 14$ 小时。
- 完成旅程，得到实际时间值 y , e.g. $y = 16$ 小时
- Loss: $L = \frac{1}{2} (q - y)^2$
- 梯度: $\frac{\partial L}{\partial \mathbf{w}} = \frac{\partial q}{\partial \mathbf{w}} \cdot \frac{\partial L}{\partial q} = (q - y) \cdot \frac{\partial Q(\mathbf{w})}{\partial \mathbf{w}}$.
- 梯度下降: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \cdot \frac{\partial L}{\partial \mathbf{w}} \big|_{\mathbf{w}=\mathbf{w}_t}$.

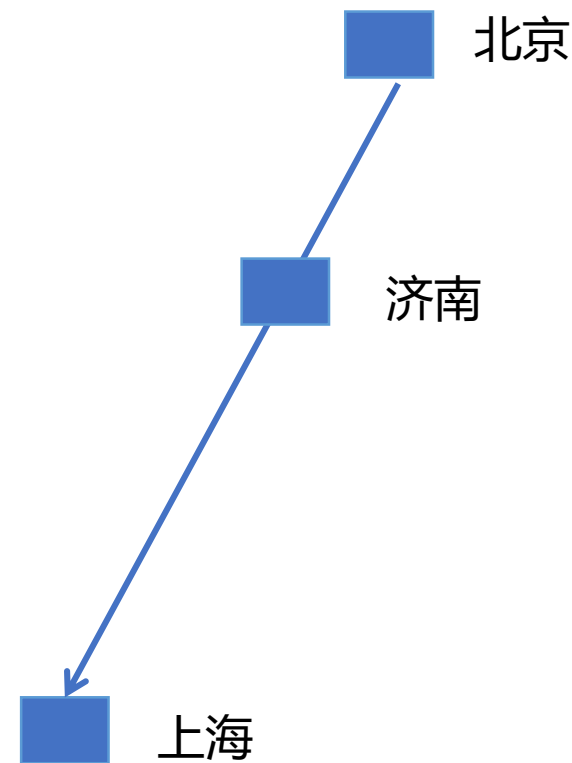


如何训练DQN--TD learning

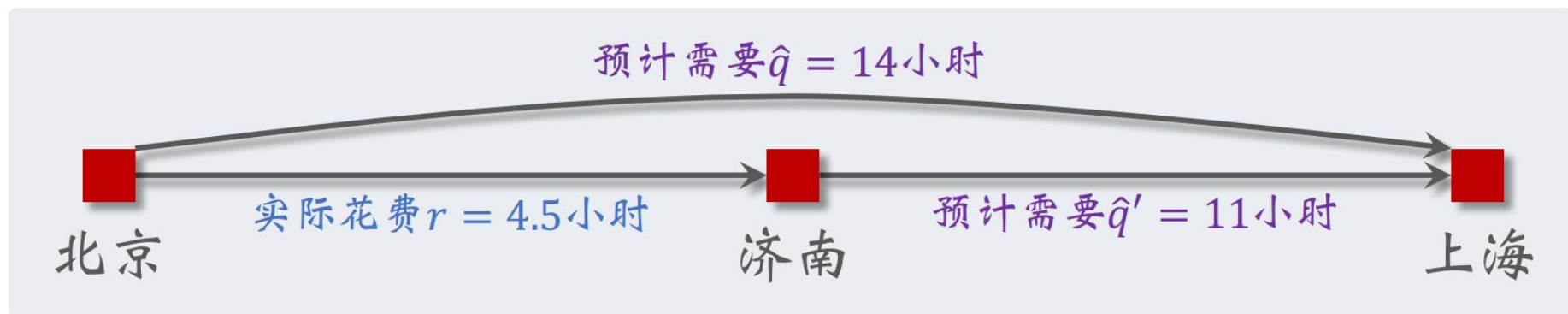
- temporal difference (TD) learning: 时间差分算法
- 举例：假设我是个用户，我要从北京驾车去上海。
- 模型 $Q(w)$ 估计总车程：例如是 14 小时：

问题：如何更新模型？

- 我是否在旅行结束之前来更新模型？
- 我没有完成旅途，我是否可以用到达济南的数据帮助训练模型呢？



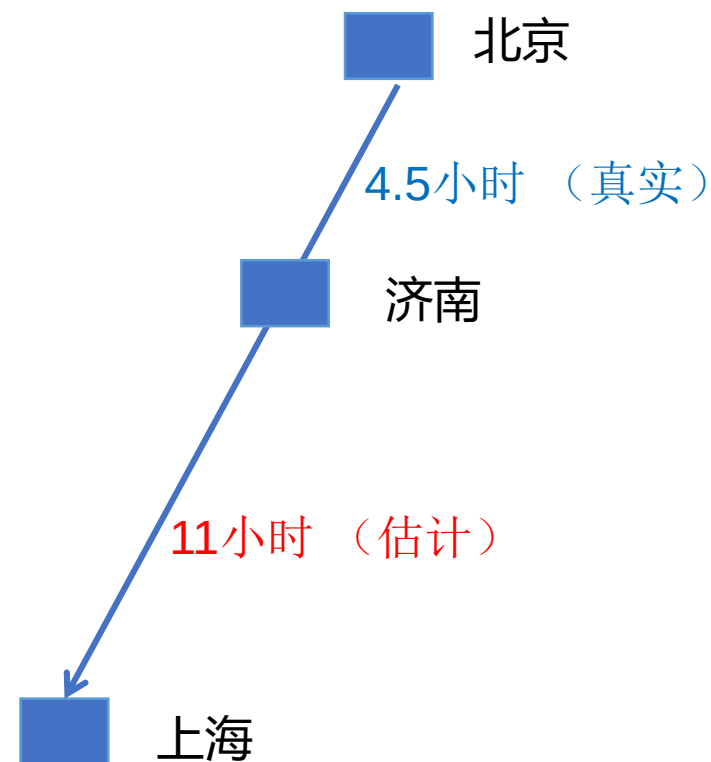
如何训练DQN--TD learning



- $Q(w)$ 模型估计：北京到上海**14小时**（估计）
- 我到达济南,实际花费：北京到济南：**4.5小时**（真实）
- 现在模型更新它的估计：
 - 济南到上海：**11小时**（估计）

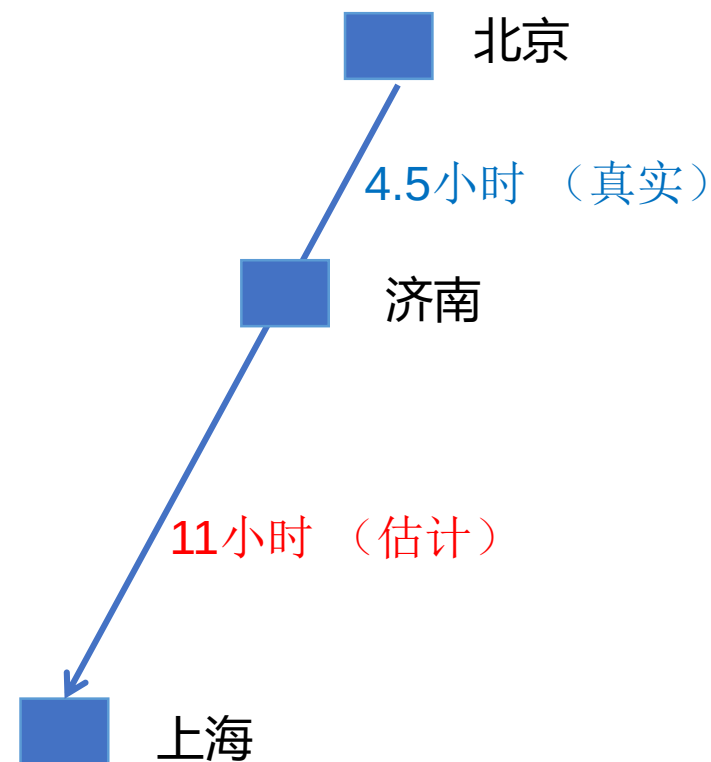
如何训练DQN--TD learning

- 模型估计: $Q(w) = 14$ 小时
- 更新估计: $4.5 + 11 = 15.5$ 小时
↓
TD target
- TD 目标 $y = 15.5$ 比它最初的预测14更可靠
- Loss: $L = \frac{1}{2} (Q(w) - y)^2$
- 梯度: $\frac{\partial L}{\partial w} = \underbrace{(1000 - 900)}_{\text{TD error}} \cdot \frac{\partial Q(w)}{\partial w}$
- 梯度下降: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \cdot \frac{\partial L}{\partial \mathbf{w}} \Big|_{\mathbf{w}=\mathbf{w}_t}$



如何训练DQN--TD learning

- 模型估计: $Q(w) = 14$ 小时
- 更新估计: $4.5 + 11 = 15.5$ 小时
↓
TD target
- TD 目标 $y = 15.5$ 比它最初的预测14更可靠
- Loss: $L = \frac{1}{2} (Q(w) - y)^2$
- 梯度: $\frac{\partial L}{\partial w} = \underbrace{(1000 - 900)}_{\text{TD error}} \cdot \frac{\partial Q(w)}{\partial w}$
- 梯度下降: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \cdot \frac{\partial L}{\partial \mathbf{w}} \Big|_{\mathbf{w}=\mathbf{w}_t}$



如何用TD训练DQN ?

- 在旅行例子中，我们有这样的方程：

$$T_{\text{北京} \rightarrow \text{上海}} \approx T_{\text{北京} \rightarrow \text{济南}} + T_{\text{济南} \rightarrow \text{上海}}$$

模型的估计 真实时间 模型的估计

- 在深度强化学习中：

$$Q(s_t, a_t; \mathbf{w}) \approx r_t + \gamma \cdot Q(s_{t+1}, a_{t+1}; \mathbf{w}).$$

如何用TD训练DQN ?

- 折扣回报: $U_t = \sum_{k=t}^n \gamma^{k-t} \cdot R_k$ $U_{t+1} = \sum_{k=t+1}^n \gamma^{k-t-1} \cdot R_k$

$$U_t = R_t + \underbrace{\gamma \cdot \sum_{k=t+1}^n \gamma^{k-t-1} \cdot R_k}_{= U_{t+1}}$$

- TD learning for DQN

- DQN's output, $Q(s_t, a_t; \mathbf{w})$, is an estimate of U_t .
- DQN's output, $Q(s_{t+1}, a_{t+1}; \mathbf{w})$, is an estimate of U_{t+1} .

- Thus,
$$\underbrace{Q(s_t, a_t; \mathbf{w})}_{\text{Prediction}} \approx \underbrace{r_t + \gamma \cdot Q(s_{t+1}, a_{t+1}; \mathbf{w})}_{\text{TD target}}.$$

如何用TD训练DQN ?

- Prediction: $Q(s_t, a_t; w_t)$
- TD target:

$$\begin{aligned} y_t &= r_t + \gamma \cdot Q(s_{t+1}, a_{t+1}; w_{t+1}) \\ &= r_t + \gamma \cdot \max_a Q(s_{t+1}, a; w_{t+1}) \end{aligned}$$

- Loss: $L_t = \frac{1}{2} [Q(s_t, a_t; \mathbf{w}) - y_t]^2$.
- Gradient descent: $\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \cdot \frac{\partial L_t}{\partial \mathbf{w}} \Big|_{\mathbf{w}=\mathbf{w}_t}$.

本次课的主要内容



北京邮电大学
Beijing University of Posts and Telecommunications

强化学习基本概念

基于值的强化学习

基于策略的强化学习

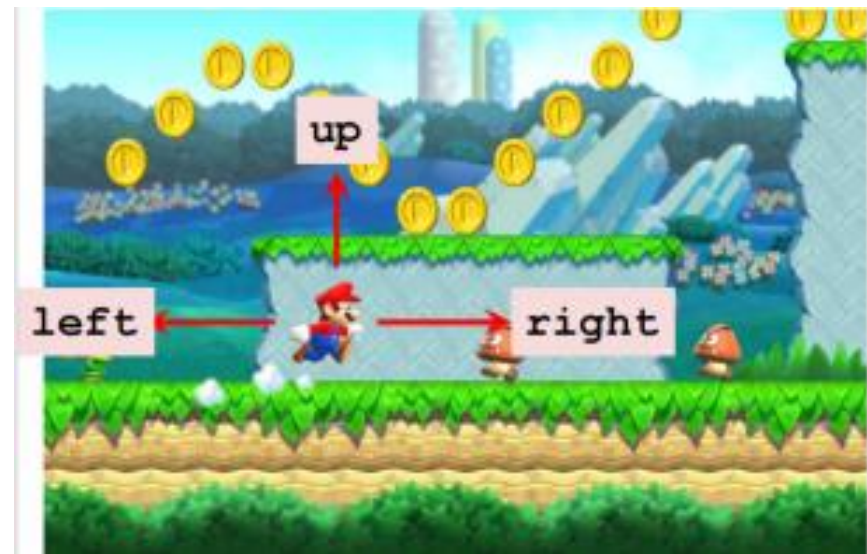
法

策略函数 $\pi(a|s)$

- 策略函数 π 是个条件概率密度函数
- 输入状态 s
- 输出所有的动作的概率，比如

$$\begin{aligned}\pi(\text{left}|s) &= 0.2, \\ \pi(\text{right}|s) &= 0.1, \\ \pi(\text{up}|s) &= 0.7.\end{aligned}$$

- 基于这个分布，我们通过随机抽样确定动作 a



例子-Grid world

- 引用一个经典的Grid World例子。假设有一个 $4 \times 4 = 16$ 的网格世界。其中左上角和右下角格子是终止格子。每次只能向上下左右移动一个格子，如果在边界格子往外走，则会直接移动回到之前的边界格。
- 每次移动的奖励 R 均为-1，终止格子奖励是0，价值也固定为0。
- 由于这里每次移动的下一格子都是固定的，因此状态转化概率都是 $P=1$ 。衰减因子定义为 $\gamma = 1$ 。
- 这里给定的初始策略是**随机策略**，每个格子向其周围4个格子移动的概率都是0.25。

	1	2	3
4	5	6	7
8	9	10	11
12	13	14	



动作

随机策略 VS 贪心策略

■ 状态值函数

$$v^{\pi}(s_t) = \sum_{a_t} \pi(a_t|s_t) \sum_{s_{t+1}} p(s_{t+1}|s_t, a_t) [r_{a_t}^{s_{t+1}} + \gamma v^{\pi}(s_{t+1})]$$

此例子中 π 均为1/4, p 均为1, 奖励函数均为 -1, $\gamma = 1$

■ 除了到达终点外, 状态值函数为

$$V_{\pi}(s_t) = \sum_1^4 \frac{1}{4} [-1 + V_{\pi}(s_{t+1})]$$



随机策略 VS 贪心策略

➤ 随机策略

$k=0$

0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0
0.0	0.0	0.0	0.0

$k=1$

0.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	-1.0
-1.0	-1.0	-1.0	0.0

$k=2$

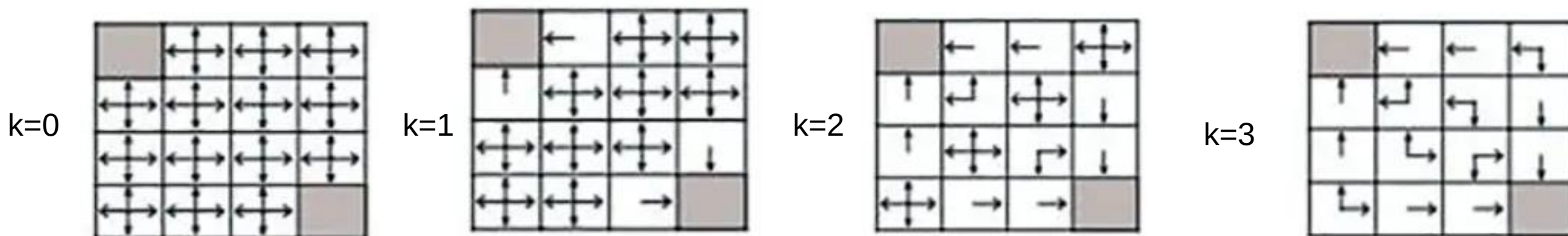
0.0	-1.7	-2.0	-2.0
-1.7	-2.0	-2.0	-2.0
-2.0	-2.0	-2.0	-1.7
-2.0	-2.0	-1.7	0.0

$k=3$

0.0	-2.4	-2.9	-3.0
-2.4	-2.9	-3.0	-2.9
-2.9	-3.0	-2.9	-2.4
-3.0	-2.9	-2.4	0.0

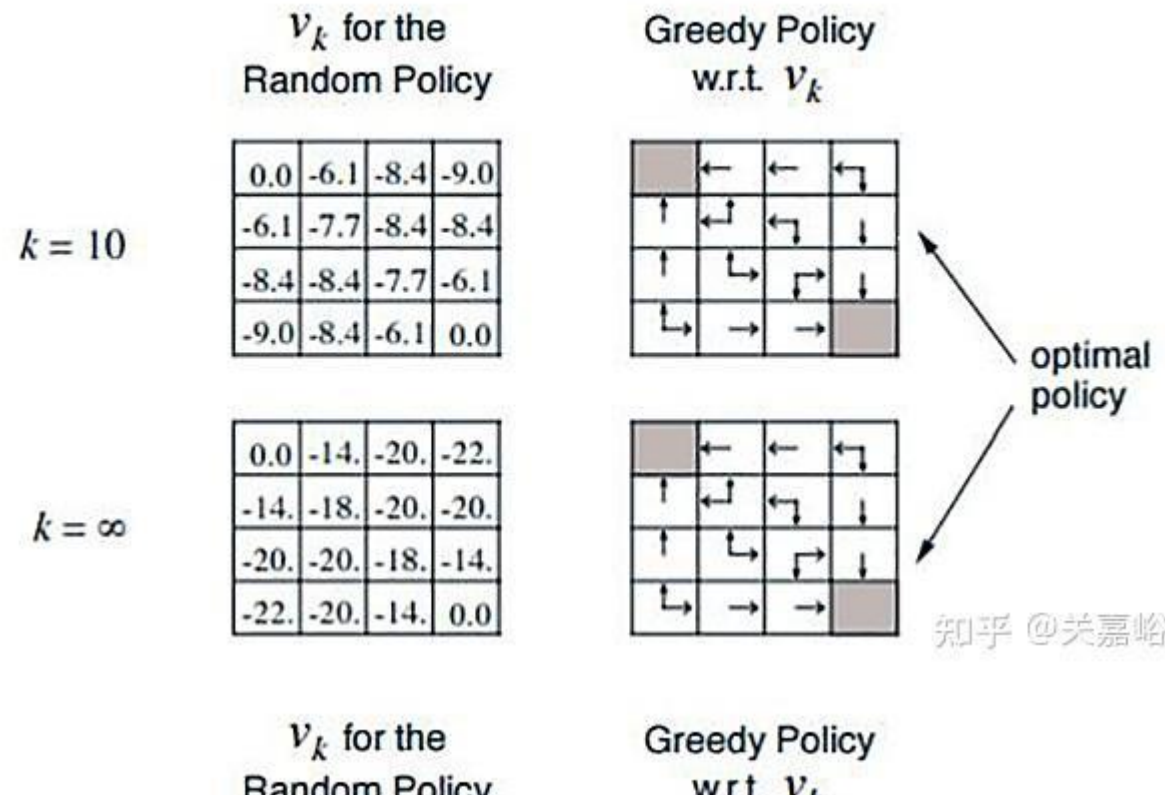
➤ 策略改进-贪心策略

$$\pi(s) = \arg \max_a Q(s, a)$$



随机策略 VS 贪心策略

- 从图中可知，当 $k=\infty$ 时， $v(3,1)$ 格子上下左右的状态值分别是-20， x ， -22， -14。因为-14最大，因此改进的策略应该选-14， 那么最优动作就是向右。
- 但是，得到的新策略也未必是最优的，如果不是，可以返回策略评估，继续用贝尔曼方程迭代，进行策略评估，如此反复进行，直到得到收敛的策略 π^*



策略迭代 VS 值迭代



策略迭代算法

输入: MDP 五元组: $\mathcal{S}, \mathcal{A}, P, R, \gamma$;

1 初始化: $\forall s, \forall a, \pi(a|s) = \frac{1}{|\mathcal{A}|}$; 【随机策略, $|\mathcal{A}|$ 是动作总个数】

2 repeat

 // 策略评估

 【计算所有状态 s 的状态值函数】

3 repeat

4 根据贝尔曼方程 $V^\pi(s) = E_a E_{s'} [R(s, a, s') + \gamma V^\pi(s')]$

 计算 $V^\pi(s), \forall s$;

5 until $\forall s, V^\pi(s)$ 收敛;

 // 策略改进

 【计算所有状态 s 所有动作 a 的值函数】

6 根据公式 $Q^\pi(s, a) = E_{s'} [R(s, a, s') + \gamma V^\pi(s')]$

 计算 $Q(s, a)$;

7 $\forall s, \pi(s) = \arg \max_a Q(s, a)$; 【每个状态 s 取动作值函数最大的动作】

8 until $\forall s, \pi(s)$ 收敛;

输出: 策略 π

值迭代算法

输入: MDP 五元组: $\mathcal{S}, \mathcal{A}, P, r, \gamma$;

1 初始化: $\forall s \in \mathcal{S}, V(s) = 0$;

2 repeat

3 $\forall s, V(s) \leftarrow \max_a E_{s'} [r(s, a, s') + \gamma V(s')]$;

4 until $\forall s, V(s)$ 收敛;

5 根据公式 $Q^\pi(s, a) = E_{s'} [r(s, a, s') + \gamma V^\pi(s')]$

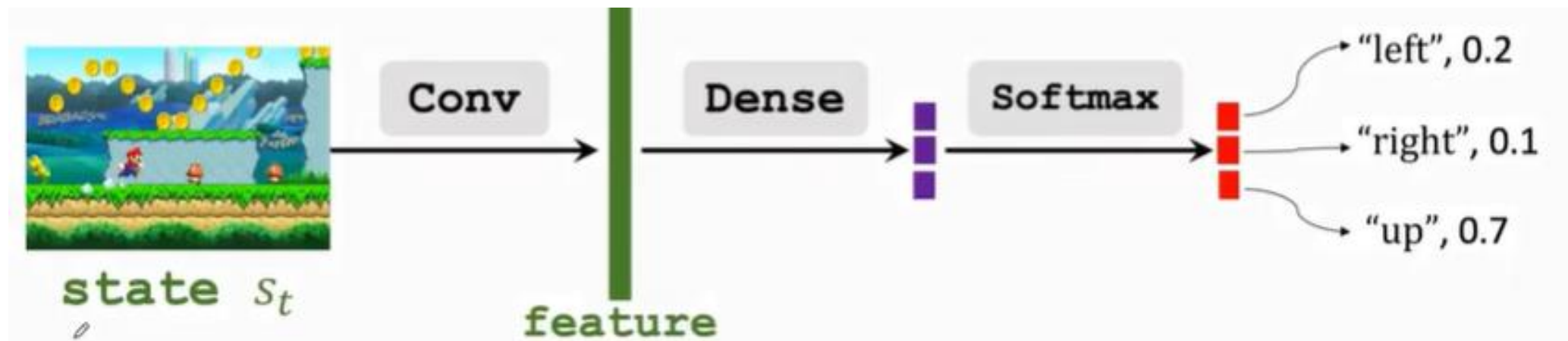
 计算 $Q(s, a)$;

6 $\forall s, \pi(s) = \arg \max_a Q(s, a)$;

输出: 策略 π

策略网络 $\pi(a|s; \theta)$?

- **问题：**当动作空间很大或连续时，上述基于表格方法会有什么问题？
- **策略网络：**一种参数化的策略表示，用神经网络 $\pi(a|s; \theta)$ 近似策略函数 $\pi(a|s)$ ， θ 为神经网络参数



$$\sum_{a \in \mathcal{A}} \pi(a|s; \theta) = 1.$$

策略网络 $\pi(a|s; \theta)$?

- **问题：**讨论为什么对于复杂或连续动作空间的问题，直接参数化策略可能是更好的选择。



注：它直接优化策略本身而不是间接通过价值函数

策略网络是如何学习呢？

➤ 状态值函数

$$V_{\pi}(s_t) = E_A[Q_{\pi}(s_t, A)] = \sum_a \pi(a|s_t) \cdot Q_{\pi}(s_t, a)$$

➤ 近似状态值函数

- 采用策略网络 $\pi(a|s_t; \theta)$ 近似策略函数 $\pi(a|s_t)$
- $V_{\pi}(s_t)$ 可以近似为

$$V_{\pi}(s_t; \theta) = \sum_a \pi(a|s_t; \theta) \cdot Q_{\pi}(s_t, a)$$

➤ 基于策略学习： 学习 θ 最大化目标函数： $J(\theta) = E_S(V(S; \theta))$

➤ 如何优化 θ ? 策略梯度下降

- 观测状态 s
- 更新策略： $\theta_{\text{new}} \leftarrow \theta_{\text{now}} + \beta \cdot \nabla_{\theta} J(\theta_{\text{now}}).$ β 是学习率



谢谢！